Algorithms

Week 12 – Graph Algorithms II

Korea University Sejong Campus, Dept. of Computer Science & Software

Part 1. Shortest Paths – Single Source

Dijkstra and Bellman-Ford

Shortest Paths – Overview

- **Condition**: Weighted directed graph G = (V, E) with weight function w
  - An undirected edge (u, v) can be treated as two directed edges (u, v) and (v, u)
- **Shortest path** between two vertices: the path whose total edge weight is **minimum**
- If a **negative-weight cycle** is reachable from the source, shortest paths are **undefined**

Two categories:

Category	Problem	Algorithm
Single-source	Shortest paths from one source r to all other vertices	Dijkstra, Bellman-Ford
All-pairs	Shortest paths between every pair of vertices	Floyd-Warshall

Key Property 1 – Optimal Substructure

> **Subpaths of shortest paths are also shortest paths.**

```
Source ──[5]──> Intermediate ──[3]──> Goal
         ↑                       ↑
     shortest                 shortest
     to here                  from here
```

If the shortest path from S to G passes through node M with cost 5 + 3 = 8, then:

- The subpath from S to M (cost 5) is the shortest path from S to M
- The subpath from M to G (cost 3) is the shortest path from M to G

This property enables **dynamic programming** and **greedy** approaches.

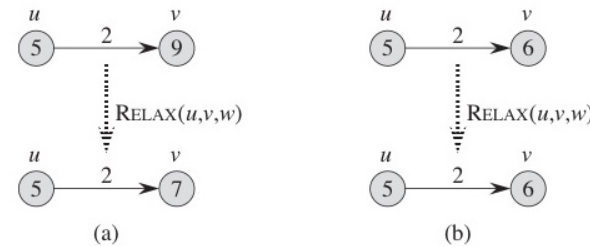
Key Property 2 – Relaxation

Relaxation = updating the shortest distance estimate when a shorter path is found.

```
RELAX(u, v, w):
    if d[u] + w(u, v) < d[v] then
        d[v] ← d[u] + w(u, v)
        prev[v] ← u
```

- **(a)** d[u]+w = 5+2 = 7 < 9 = d[v] → update d[v] = 7
- **(b)** d[u]+w = 5+2 = 7 >= 6 = d[v] → no change

Every shortest-path algorithm uses relaxation as its core operation.



Dijkstra's Algorithm – Idea

- **Greedy** strategy: always pick the unvisited vertex with the **smallest** d value
- Maintains a set S of vertices whose shortest distances are **finalized**
- **Requirement**: all edge weights must be **non-negative** (w >= 0)
  - Because adding more edges can only **increase** the total distance (greedy correctness)

Key insight: When we extract the minimum-distance vertex u from V - S, we can guarantee d[u] is already optimal – no future relaxation can improve it.

Dijkstra's Algorithm – Pseudocode

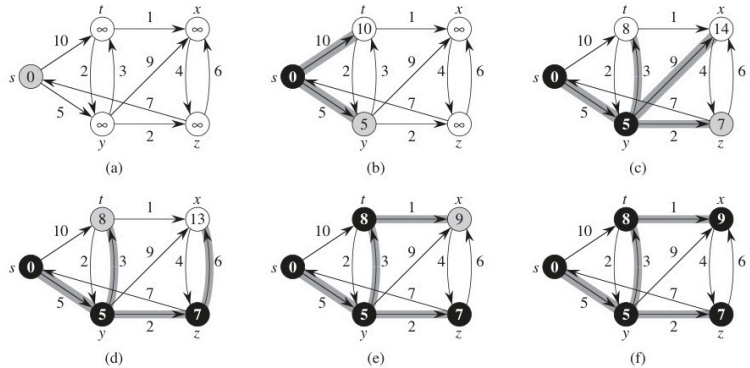
```
Dijkstra(G, r):
    S ← empty set
    for each u in V:
        d[u] ← infinity
    d[r] ← 0

    while S ≠ V:                          // n iterations
        u ← extractMin(V - S, d)          // vertex with smallest d
        S ← S union {u}
        for each v in Adj(u):             // relax neighbors
            if v in (V - S) and d[u] + w(u,v) < d[v]:
                d[v] ← d[u] + w(u,v)
                prev[v] ← u
```

Time complexity: O(|E| log |V|) with a binary min-heap (priority queue)

- Each vertex extracted once: O(|V| log |V|)
- Each edge relaxed once, decrease-key: O(|E| log |V|)
- Total: **O(|E| log |V|)**

Dijkstra – Step-by-Step Example

Step	Extracted	d[s]	d[t]	d[x]	d[y]	d[z]
Init	–	**0**	inf	inf	inf	inf
(b)	s	0	10	inf	5	inf
©	y	0	8	14	5	7
(d)	z	0	8	13	5	7
(e)	t	0	8	9	5	7
(f)	x	0	8	9	5	7



The predecessor array `prev[]` lets us reconstruct the actual shortest path. Bold edges show the shortest-path tree.

Why Dijkstra Fails with Negative Weights

```
A ---1⟶ B ---2⟶ C
|                  ^
+————————0————————+
```

Edges: A->B (1), A->C (0), B->C (-2). Source = A.

- Init: d[A]=0, d[B]=inf, d[C]=inf
- Extract A (d=0): relax A->B: d[B]=1. relax A->C: d[C]=0
- Extract C (d=0, **finalized!**): no outgoing edges from C
- Extract B (d=1): relax B->C: 1+(-2) = -1 < 0, but C is already finalized!
- Dijkstra outputs d[C]=0, but the true shortest path is A->B->C = 1+(-2) = **-1**

The greedy assumption – "extracting the minimum means the distance is final" – relies on the fact that adding more edges can only **increase** cost. **Negative weights violate this assumption** because a longer path (more edges) can end up cheaper.

This is why we need **Bellman-Ford** for graphs with negative weights.

Bellman-Ford Algorithm – Pseudocode

```
BellmanFord(G, r):
    for each u in V:
        d[u] ← infinity
    d[r] ← 0

    for i ← 1 to |V| - 1:                // Phase 1: relax |V|-1 times
        for each edge (u, v) in E:
            if d[u] + w(u,v) < d[v]:
                d[v] ← d[u] + w(u,v)
                prev[v] ← u

    for each edge (u, v) in E:            // Phase 2: detect negative cycle
        if d[u] + w(u,v) < d[v]:
            output "No solution (negative cycle)"
```

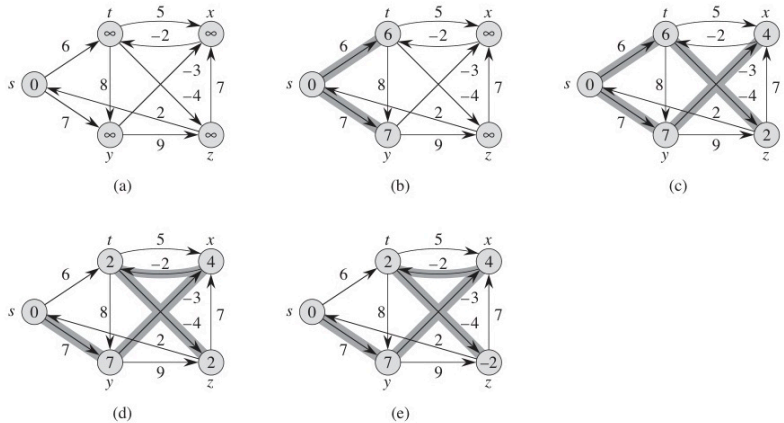
Key ideas:

- Allows **negative weights** (edges can decrease distance)
- After |V|-1 iterations, all shortest paths are found (a shortest path has at most |V|-1 edges)
- If distances **still decrease** after |V|-1 rounds, a **negative-weight cycle** exists

Bellman-Ford – Step-by-Step Example

Graph with negative edges, source = s:

Pass	d[s]	d[t]	d[x]	d[y]	d[z]
Init	**0**	inf	inf	inf	inf
(b) i=1	0	6	inf	7	inf
© i=2	0	6	4	7	2
(d) i=3	0	2	4	7	-2
(e) i=4	0	2	4	7	-2



After Phase 2: no further decrease → **no negative cycle**.

Bold edges show the shortest-path tree.

Bellman-Ford as Dynamic Programming

Define: d_t^k = shortest distance from source r to vertex t using **at most k edges**

Base case:

- d_r^0 = 0
- d_t^0 = infinity for all t != r

Recurrence:

```
d_v^k = min over all edges (u,v) { d_u^(k-1) + w(u,v) }
```

```
u1 --w1⟶ v
u2 --w2⟶ v     d_v^k = min { d_u1^(k-1)+w1,
u3 --w3⟶ v                   d_u2^(k-1)+w2,
u4 --w4⟶ v                   d_u3^(k-1)+w3,
                              d_u4^(k-1)+w4 }
```

Goal: d_t^(n-1) for all vertices t (a shortest path uses at most n-1 edges)

Single-Source Algorithms – Comparison

Property	Dijkstra	Bellman-Ford
Negative weights	Not allowed	Allowed
Negative cycle detection	No	Yes
Strategy	Greedy	Dynamic Programming
Time complexity	O(|E| log |V|)	Theta(|V| * |E|)
Data structure	Priority queue (min-heap)	Simple edge list
When to use	Non-negative weights	Negative weights possible

Part 2. All-Pairs Shortest Paths and SCCs

Floyd-Warshall, DAG Shortest Paths, Strongly Connected Components

Floyd-Warshall – Motivation

Problem: Find shortest paths between **all pairs** of vertices.

- **Dijkstra from every vertex**: O(|V| * |E| log |V|) – expensive for dense graphs
- **Floyd-Warshall**: Theta(|V|^3) – simpler, handles negative weights (but no negative cycles)

Applications:

- Road atlas (precompute all city-to-city distances)
- Navigation systems
- Network routing

	Dijkstra (single-source)	Floyd-Warshall (all-pairs)
Computes	Source -> all other vertices	Every vertex -> every vertex
Input	Adjacency list	Adjacency matrix

Floyd-Warshall – Input Matrix

Uses an **adjacency matrix** W where:

- W[i][j] = weight of edge (i, j) if it exists
- W[i][j] = infinity if no direct edge
- W[i][i] = 0

Example (CLRS p.341):

	1	2	3	4	5
1	0	3	8	inf	-4
2	inf	0	inf	1	7
3	inf	4	0	inf	inf
4	2	inf	-5	0	inf

Floyd-Warshall — Node-Based Formulation

Define: **d_ij^(k)** = shortest path from i to j using only intermediate vertices from {v1, v2, …, vk}

```
i ——[intermediate vertices in {1, … ,k-1}]——→ j
                    OR
i ——→ k ——→ j
 (d_ik^(k-1))  (d_kj^(k-1))
```

Recurrence:

```
d_ij^(0) = w_ij                                          (direct edges only)
d_ij^(k) = min { d_ij^(k-1),  d_ik^(k-1) + d_kj^(k-1) }  (k ≥ 1)
```

Two choices at each step:

1. **Don't use vertex k** as intermediate: d_ij^(k-1)
2. **Use vertex k** as intermediate: d_ik^(k-1) + d_kj^(k-1)

Take the minimum.

Floyd-Warshall – Pseudocode

```
FloydWarshall(G):
    // Initialize D^(0)
    for i ← 1 to n:
        for j ← 1 to n:
            d_ij^(0) ← w_ij

    // Main computation
    for k ← 1 to n:                    // intermediate vertex set {1, … ,k}
        for i ← 1 to n:                // source vertex
            for j ← 1 to n:            // destination vertex
                d_ij^(k) ← min { d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1) }
```

Time complexity: Theta(|V|^3)

- Total subproblems: Theta(|V|^3)
- Each subproblem: Theta(1)

Floyd-Warshall – Step-by-Step (k=0)

D^(0) = W (direct edges only, no intermediate vertices)

	1	2	3	4	5
1	0	3	8	inf	-4
2	inf	0	inf	1	7
3	inf	4	0	inf	inf
4	2	inf	-5	0	inf
5	inf	inf	inf	6	0

This is the base case: d_ij^(0) = w_ij.

k = 0 means no intermediate vertices are allowed – only direct edges.

Floyd-Warshall — Step-by-Step (k=1)

D^(1): allow vertex 1 as intermediate.

d_ij^(1) = min{ d_ij^(0), d_i1^(0) + d_1j^(0) }

Example: d_42^(1) = min{ d_42^(0), d_41^(0) + d_12^(0) } = min{ inf, 2 + 3 } = **5**

Example: d_45^(1) = min{ d_45^(0), d_41^(0) + d_15^(0) } = min{ inf, 2 + (-4) } = **-2**

	1	2	3	4	5
1	0	3	8	inf	-4
2	inf	0	inf	1	7
3	inf	4	0	inf	inf
4	2	**5**	-5	0	**-2**

Floyd-Warshall — Step-by-Step (k=2)

D^(2): allow vertices {1, 2} as intermediates.

d_ij^(2) = min{ d_ij^(1), d_i2^(1) + d_2j^(1) }

Example: d_14^(2) = min{ d_14^(1), d_12^(1) + d_24^(1) } = min{ inf, 3 + 1 } = **4**

Example: d_34^(2) = min{ d_34^(1), d_32^(1) + d_24^(1) } = min{ inf, 4 + 1 } = **5**

	1	2	3	4	5
1	0	3	8	**4**	-4
2	inf	0	inf	1	7
3	inf	4	0	**5**	**11**
4	2	5	-5	0	-2

Floyd-Warshall — Step-by-Step (k=3)

D^(3): allow vertices {1, 2, 3} as intermediates.

d_ij^(3) = min{ d_ij^(2), d_i3^(2) + d_3j^(2) }

Example: d_42^(3) = min{ d_42^(2), d_43^(2) + d_32^(2) } = min{ 5, (-5) + 4 } = **-1**

	1	2	3	4	5
1	0	3	8	4	-4
2	inf	0	inf	1	7
3	inf	4	0	5	11
4	2	**-1**	-5	0	-2
5	inf	inf	inf	6	0

Floyd-Warshall – Final Result (D^(5))

After computing D^(4) and D^(5), we obtain the **all-pairs shortest distances**:

	1	2	3	4	5
1	0	1	-3	2	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Path reconstruction example: Shortest path 1 -> 2 (cost = 1):

Trace back using predecessor matrix:

Floyd-Warshall — Predecessor Matrix

The **predecessor matrix** Pi^(k) is computed alongside D^(k):

- pi_ij^(k) = predecessor of j on shortest path from i to j using intermediates {1,…,k}

Base case (k = 0):

```
pi_ij^(0) = i      if i ≠ j and w_ij < infinity
pi_ij^(0) = NIL    if i = j or w_ij = infinity
```

Recurrence (k >= 1):

```
pi_ij^(k) = pi_ij^(k-1)          if d_ij^(k-1) ≤ d_ik^(k-1) + d_kj^(k-1)
pi_ij^(k) = pi_kj^(k-1)          if d_ij^(k-1) > d_ik^(k-1) + d_kj^(k-1)
```

To reconstruct path from i to j: follow predecessors backward from j until reaching i.

Floyd-Warshall — Edge-Based Formulation

An alternative way to think about Floyd-Warshall (CLRS p.341-342):

Define: **l_ij^(m)** = shortest path from i to j using **at most m edges**

Recurrence:

```
l_ij^(m) = min over 1≤k≤n { l_ik^(m-1) + w_kj }
```

This resembles **matrix multiplication** where:

- "multiply" = add (l_ik + w_kj)
- "add" = min

Computation: L^(1) = W, L^(2) = L^(1) "x" W, …, L^(n-1) = final result

For n = 5: L^(4) gives all-pairs shortest distances (at most n-1 = 4 edges).

Time: Theta(n^4) naively, or Theta(n^3 log n) using repeated squaring.

DAG Shortest Paths

For **Directed Acyclic Graphs** (DAGs), shortest paths can be found in **linear time**.

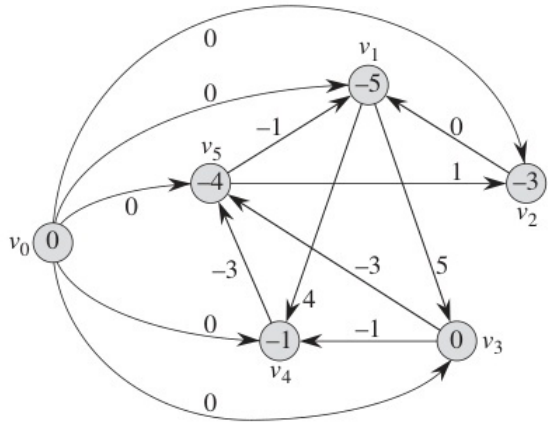
```
DAG-ShortestPath(G, r):
    for each u in V:
        d[u] ← infinity
    d[r] ← 0

    Topologically sort the vertices of G

    for each u in V (in topological order):
        for each v in Adj(u):
            if d[u] + w(u,v) < d[v]:
                d[v] ← d[u] + w(u,v)
```

Time complexity: Theta(|V| + |E|)

- Topological sort: Theta(|V| + |E|)
- Single pass over all edges: Theta(|V| + |E|)
- **Much faster** than Dijkstra or Bellman-Ford when graph is a DAG

DAG Shortest Paths – Example

Step	Process	d[v0]	d[v1]	d[v2]	d[v3]	d[v4]	d[v5]
Init	–	**0**	inf	inf	inf	inf	inf
(a)	v0	0	inf	inf	inf	**-4**	inf
(b)	v1	0	inf	inf	inf	-4	**-1**
©	v2	0	**-5**	inf	**0**	-4	-1
(d)	v3	0	-5	**-3**	0	-4	-1
(e)	v4	0	-5	-3	0	-4	-1
(f)	v5	0	-5	-3	0	-4	-1



All Shortest Path Algorithms – Summary

Algorithm	Problem	Neg. weights	Time	Technique
Dijkstra	Single-source	No	O(|E| log |V|)	Greedy
Bellman-Ford	Single-source	Yes	Theta(|V| |E|)	DP
DAG-ShortestPath	Single-source (DAG)	Yes	Theta(|V|+|E|)	Topo sort
Floyd-Warshall	All-pairs	Yes	Theta(|V|^3)	DP

How to choose:

1. DAG? Use DAG-ShortestPath (fastest)
2. No negative weights? Use Dijkstra (efficient single-source)
3. Negative weights possible? Use Bellman-Ford (detects negative cycles)
4. Need all pairs? Use Floyd-Warshall

Part 3. Strongly Connected Components

Strongly Connected Components – Definition

- A directed graph is **strongly connected** if for every pair of vertices u, v:
  - There exists a path from u to v **AND** from v to u
- A **Strongly Connected Component** (SCC) is a **maximal** subgraph that is strongly connected

```
Example:
    1 ⟶ 2 ⟶ 3 ⟶ 4
    ^     |     ^     |
    |     v     |     v
    6 ⟵ 5     8 ⟵ 7
          |           |
          v           v
          9 ⟶ 10    (dead end)

SCCs: {1,2,5,6}, {3,4,7,8}, {9}, {10}
```

Kosaraju's Algorithm – Pseudocode

```
StronglyConnectedComponents(G):

1. Run DFS on G and compute finish times f[v] for every vertex v.

2. Construct G^R (reverse all edges in G).

3. Run DFS on G^R, but process vertices in DECREASING order of f[v]
   (i.e., start from the vertex that finished LAST in step 1).

4. Each DFS tree in step 3 is one strongly connected component.
```

Time complexity: Theta(|V| + |E|)

- Step 1 (DFS on G): Theta(|V| + |E|)
- Step 2 (reverse edges): Theta(|V| + |E|)
- Step 3 (DFS on G^R): Theta(|V| + |E|)

Kosaraju's Algorithm – Example

Step 1: DFS on G, record finish times.

```
Graph G:
  1 ⟶ 2    5 ⟶ 6
  ^     |    ^     |
  |     v    |     v
  4 ⟵ 3    8 ⟵ 7

  Also: 3⟶5, 6⟶4, 8⟶9, 9⟶10, 10⟶8
```

DFS from vertex 1 (example finish-time order): f[10]=1, f[9]=2, f[8]=3, f[7]=4, f[6]=5, f[5]=6, f[3]=7, f[2]=8, f[4]=9, f[1]=10

Step 2: Reverse all edges to get G^R.

Kosaraju's Algorithm – Example (continued)

Step 3: DFS on G^R in decreasing finish-time order.

Process order: 1 (f=10), 4 (f=9), 2 (f=8), 3 (f=7), 5 (f=6), …

- Start DFS from vertex **1**: reaches {1, 4, 3, 2} – **SCC 1**
- Start DFS from vertex **5**: reaches {5, 6, 7, 8} – **SCC 2**
- Start DFS from vertex **9**: reaches {9, 10} – **SCC 3**

Result: 3 SCCs = {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10}

Kosaraju's Algorithm – Why It Works

Key insight: If u and v are in the same SCC, then:

- There exists a path u -> v in G, which means v -> u in G^R
- There exists a path v -> u in G, which means u -> v in G^R
- Therefore, u and v are also in the same SCC in G^R

Finish-time ordering ensures correctness:

- The vertex with the highest finish time in DFS(G) belongs to a "source" SCC (an SCC with no incoming edges from other SCCs)
- Starting DFS(G^R) from this vertex only reaches vertices in the same SCC
- After removing that SCC, the next highest finish time leads to the next source SCC
- This peeling process correctly identifies all SCCs

Summary

Topic	Key Points
Shortest path properties	Optimal substructure, relaxation
Dijkstra	Greedy, non-negative weights, $O(\|E\| \log \|V\|)$
Bellman-Ford	DP, negative weights OK, detects negative cycles, $\Theta(\|V\|\|E\|)$
Floyd-Warshall	All-pairs DP, $\Theta(\|V\|^3)$, node-based or edge-based formulation
DAG shortest path	Topological sort + single pass, $\Theta(\|V\|+\|E\|)$
SCCs (Kosaraju)	Two DFS passes (G then G^R), $\Theta(\|V\|+\|E\|)$

Next week: **NP-Completeness & Approximation Algorithms**

Q & A

codingchild@korea.ac.kr